

Count Fuzzer labjournal

Anna Latour

2022

Research goals

- Develop fuzzer for counting problems, including support for:
 - Exact model counting.
 - PAC counting.
 - Projected counting.
 - Weighted model counting.
 - Develop delta debugger for counting problems, including support for the problems mentioned above.
 - Identify problems of the following kinds in existing counters:
 - Errors (unexpected termination without providing a result).
 - Incorrect counts (we might have to define that for PAC counters).
 - Fuzz the preprocessor(s).
 - Generate instances with known ground truth count, especially for weighed model counting.
-

Research questions and design questions

- How to automatically generate diverse problem instances for the different counting problems?
 - How to automatically generate challenging problem instances for the different counting problems?
 - How easy/hard should these problems be?
 - How do we determine the true count?
 - How do we define an incorrect count in the case of PAC counting?
 - (How) do we account for differences in interpretation of the contribution to the model count of free variables?
 - How to efficiently minimise the instance size of bug triggering instances?
-

Experimental evaluation

We have three main experiments to perform.

The first measures how many errors and incorrect counts we find for each solver. Following [Brummayer et al., 2010]¹ we should let the fuzzer generate a fixed number of instances (e.g., 10 000) and see how many of those lead to errors and incorrect counts in the solvers on which we evaluate. Let's call this set of instances $\mathcal{T}$.

It is possible that the same bug can lead to an error in one instance, and an incorrect count in another. It is also possible that multiple instances trigger the same bug. To gain more insight in this dynamic, we can run a second experiment.

We can take a solver and solve the errors and miscounts. Then, we can for each bugfix record how many errors and miscounts are resolved by that bugfix. That way, we'll have some idea of how much overlap there is in the instances that trigger the bugs, and in the outcome of those bugs (errors and miscounts).

For example, we could take ApproxMC4 [Chakraborty et al., 2016, Soos and Meel, 2019]² as this solver. We would then take the most recent commit from *before* we started this project as the baseline version of ApproxMC4 (here we would have to take care to also use the appropriate commits for CryptoMiniSAT [Soos et al., 2009]³ and Arjun [Soos and Meel, 2021]⁴). While we develop the fuzzer, we will undoubtedly find errors and miscounts in ApproxMC4. We should properly record the instances that triggered them, and the commits in which the bugfixes were implemented. After we have run the "pre-project commit" version of ApproxMC4 on the instances in $\mathcal{T}$ for the experimental evaluation, we can run whatever version of ApproxMC4 we have by then on $\mathcal{T}$, and fix whichever bugs arise from there, again carefully recording the instances and the commits with the bugfixes.

Finally, we can then classify all the bugfixes to determine if they fix an error or a miscount, and link them to the instance(s) from $\mathcal{T}$ that triggered them. From there, we can do some reporting on how many instances trigger the same bug, and if bugfixes tend to repair solely errors or miscounts or both.

The third main experiment would evaluate the effectiveness of the delta debugger. Following [Brummayer et al., 2010], we can take the instances from $\mathcal{T}$ that trigger errors or miscounts, minimise them, and record the time it takes to minimise them and record by how much we could minimise them.

¹ R. Brummayer, F. Lonsing, and A. Biere. Automated testing and debugging of SAT and QBF solvers. In O. Strichman and S. Szeider, editors, *Theory and Applications of Satisfiability Testing - SAT 2010, 13th International Conference, SAT 2010, Edinburgh, UK, July 11-14, 2010. Proceedings*, volume 6175 of *Lecture Notes in Computer Science*, pages 44–57. Springer, 2010. DOI: 10.1007/978-3-642-14186-7_6. URL https://doi.org/10.1007/978-3-642-14186-7_6

² S. Chakraborty, K. S. Meel, and M. Y. Vardi. Algorithmic improvements in approximate counting for probabilistic inference: From linear to logarithmic sat calls. In *Proceedings of International Joint Conference on Artificial Intelligence (IJCAI)*, 7 2016; and M. Soos and K. S. Meel. Bird: Engineering an efficient cnf-xor sat solver and its applications to approximate model counting. In *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*, 1 2019

³ M. Soos, K. Nohl, and C. Castelluccia. Extending SAT solvers to cryptographic problems. In O. Kullmann, editor, *Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009, Swansea, UK, June 30 - July 3, 2009. Proceedings*, volume 5584 of *Lecture Notes in Computer Science*, pages 244–257. Springer, 2009. DOI: 10.1007/978-3-642-02777-2_24. URL https://doi.org/10.1007/978-3-642-02777-2_24

⁴ M. Soos and K. S. Meel. Arjun: An efficient independent support computation technique and its applications to counting and sampling. *CoRR*, abs/2110.09026, 2021. URL <https://arxiv.org/abs/2110.09026>

Relevant Literature

- [Brummayer and Biere, 2009] *Fuzzing and Delta-Debugging SMT Solvers*.
 - [Brummayer et al., 2010] *Automated Testing and Debugging of SAT and QBF Solvers*.
-

Counters to evaluate

- D4.⁵
- DPMC.⁶
- GANAK [Sharma et al., 2019].
- GPMC.⁷
- ExactMC [Lai et al., 2021].
- SharpSAT [Thurley, 2006].
- SharpSAT-TD.⁸
- ApproxMC4 [Chakraborty et al., 2016, Soos and Meel, 2019].

⁵ Available at github.com/crillab/d4.

⁶ Available at github.com/vardigroup/DPMC.

⁷ Available at www.trs.cm.is.nagoya-u.ac.jp/projects/PMC.

⁸ Available at github.com/Laakeri/sharpsat-td.

We should also fuzz the B+E [Lagniez et al., 2016, 2020] and Arjun [Soos and Meel, 2021] preprocessors in combination with the different counters.

READING

Summary of [Brummayer and Biere, 2009]

Title: *Fuzzing and Delta-Debugging SMT Solvers*

Reading date: 2022-10-31

The authors present fuzzers and delta-debuggers for CNF and QBF formulae.

Specifically, they present three fuzzers based on *grammar-based black-box fuzzing*: CNFuzz, FuzzSAT, BlocksQBF and QBFuzz.

While the authors had access to a random 3SAT generator (3SAT-Gen), they decided to create CNFuzz. This random instance generator incorporates structure in the generated CNFs, which better reflects reality and has a better chance of triggering bugs in the SAT

solvers, because they heavily rely on structure for solving the instances. The FuzzSAT instance generator is even more structured, translating random DAGs into CNF.

The authors also implemented BLOCKSQBF, which generates random QBFs based on a model from the literature that they adapted for this purpose. To increase the diversity of the generated instances, they also implement QBFuzz, which has a different approach to selecting literals to incorporate in clauses.

The authors present experiments where they take the three different SAT fuzzers, have them each generate 10 000 instances, and record the numbers of *errors* (early termination with no result), *incorrects* (solver reporting that CNF is UNSAT while it is SAT or vice versa), and *invalid models* (where the solver correctly identifies a CNF as SAT, but returns an assignment that is not in fact a model), for different solvers.

They perform similar experiments for the two different QBF fuzzers.

Their other contribution is on delta debugging. They implement two delta debuggers for SAT, *cnfdd* and the multithreaded version *mtcnfdd*, and one delta debugger for QBF, *qbidd*. Then, they took the generated instances that triggered bugs in the solvers, and used the delta debuggers to minimise those instances to the smallest size where they would still trigger the bugs on those solvers. They found that they can often reduce the size by over 85%, the only exception being a SAT solver that contained bugs that only showed up after many propagations, and hence needed somewhat larger formulae. Instance size was measured in file size. They also report minimisation time.

JOURNAL

Mon 31 Oct 2022: Anna & Mate

Brainstorm about scope of the project, instigated by the question of how we should keep track of the bugs that we identify.

Question: what could be our scientific contribution?

[Brummayer et al., 2010]'s contribution was in part in the different CNF generators that they built for fuzzing SAT solvers and QBF solvers. We could do something similar for model counting.

At first glance, it seems that we should be able to simply reuse the CNF generators that [Brummayer et al., 2010] built. However, there are some subtleties in our case.

First of all, we need a ground truth. With SAT solving, one can

verify the correctness of a returned model. For model counting, the counts are harder to verify. For weighted model counting in particular, we also have to deal with the precision of the floats used in the counter.

Hence, we see a possible scientific contribution in finding an intelligent way of generating diverse test instances for (weighted) model counting.

One approach could be to encode *Bayesian networks* (BNs) into CNF, and use a python package or something to obtain ground truths on the probabilities that we encode. We could add some diversity by encoding them using different encoding schemes. Here are a few:

Dar02 Intuitive, straightforward approach proposed in [Darwiche, 2002]⁹. For each variable, the weights of the positive and the negative literal add up to 1.

SBK05 Proposed by [Sang et al., 2005]¹⁰ as more compact alternative to **Dar02**. Less intuitive, but literal weights also add up to 1.

CD05 Proposed by [Chavira and Darwiche, 2005]¹¹ as an encoding to take advantage of local structure in the BN. It also uses shared variables. The idea is to have a single variable for each weight in the problem, to limit the size of the encoding. Additionally, [Chavira and Darwiche, 2005] implements an optimisation in which it uses clauses where exactly one literal must be true. The consequence, however, is that this encoding is not compatible with standard DPLL-based counters.

CD06 Proposed by [Chavira and Darwiche, 2006]¹² as an encoding that should work particularly well for DPLL-based solvers, since it facilitates the discovery of components. Uses prime implicants to minimise the encoding. Since this encoding is based on the **CD05** encoding, it is also not compatible with standard DPLL solvers.

BKLM16 This encoding is proposed by [Bart et al., 2016]¹³ and is based on the **CD06** encoding. However: they made it compatible with standard DPLL-based solvers.

We are currently not aware of any other encodings, but should check the most recent literature.

We could generate different Bayesian networks, encode them with different encodings, and make sure that the weights are diverse too (including extremely high and low weights), to challenge the counters.

⁹ A. Darwiche. A logical approach to factoring belief networks. In D. Fensel, F. Giunchiglia, D. L. McGuinness, and M. Williams, editors, *Proceedings of the Eighth International Conference on Principles and Knowledge Representation and Reasoning (KR-02)*, Toulouse, France, April 22-25, 2002, pages 409–420. Morgan Kaufmann, 2002.

¹⁰ T. Sang, P. Beame, and H. A. Kautz. Performing bayesian inference by weighted model counting. In M. M. Veloso and S. Kambhampati, editors, *Proceedings, The Twentieth National Conference on Artificial Intelligence and the Seventeenth Innovative Applications of Artificial Intelligence Conference*, July 9-13, 2005, Pittsburgh, Pennsylvania, USA, pages 475–482. AAAI Press / The MIT Press, 2005. URL <http://www.aaai.org/Library/AAAI/2005/aaai05-075.php>

¹¹ M. Chavira and A. Darwiche. Compiling bayesian networks with local structure. In L. P. Kaelbling and A. Safrafiotti, editors, *IJCAI-05, Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence*, Edinburgh, Scotland, UK, July 30 - August 5, 2005, pages 1306–1312. Professional Book Center, 2005. URL <http://ijcai.org/Proceedings/05/Papers/0931.pdf>

¹² M. Chavira and A. Darwiche. Encoding cnfs to empower component analysis. In A. Biere and C. P. Gomes, editors, *Theory and Applications of Satisfiability Testing - SAT 2006*, 9th International Conference, Seattle, WA, USA, August 12-15, 2006, *Proceedings*, volume 4121 of *Lecture Notes in Computer Science*, pages 61–74. Springer, 2006. DOI: 10.1007/11814948_9. URL https://doi.org/10.1007/11814948_9

¹³ A. Bart, F. Koriche, J. Lagniez, and P. Marquis. An improved CNF encoding scheme for probabilistic inference. In G. A. Kaminka, M. Fox, P. Bouquet, E. Hüllermeier, V. Dignum, F. Dignum, and F. van Harmelen, editors, *ECAI 2016 - 22nd European Conference on Artificial Intelligence*, 29 August-2 September 2016, The Hague, The Netherlands - Including Prestigious Applications of Artificial Intelligence (PAIS 2016), volume 287 of *Frontiers in Artificial Intelligence*

Mon 31 Oct 2022: Anna's afterthoughts

We could try to use a *binary decision diagram* (BDD) library to help us construct problem instances. The idea would be to build a CNF by generating (random or somehow structured) clauses, and adding them to the formula one by one. We would construct the BDD while we are doing it. BDD libraries like (the Python interface of) CUDD provide functionality for retrieving the CNF afterwards.¹⁴ As we are building the BDD incrementally, we can keep track of the model count of the CNF that it represents, by doing upward counting sweeps of the BDD. That way, we can steer towards, *e.g.*, CNFs with a very small model count compared to its number of clauses or its number of variables.

¹⁴ See

web.mit.edu/sage/export/tmp/y/usr/share/doc/poly

References

- A. Bart, F. Koriche, J. Lagniez, and P. Marquis. An improved CNF encoding scheme for probabilistic inference. In G. A. Kaminka, M. Fox, P. Bouquet, E. Hüllermeier, V. Dignum, F. Dignum, and F. van Harmelen, editors, *ECAI 2016 - 22nd European Conference on Artificial Intelligence, 29 August-2 September 2016, The Hague, The Netherlands - Including Prestigious Applications of Artificial Intelligence (PAIS 2016)*, volume 285 of *Frontiers in Artificial Intelligence and Applications*, pages 613–621. IOS Press, 2016. DOI: 10.3233/978-1-61499-672-9-613. URL <https://doi.org/10.3233/978-1-61499-672-9-613>.
- R. Brummayer and A. Biere. Fuzzing and delta-debugging smt solvers. In *Proceedings of the 7th International Workshop on Satisfiability Modulo Theories, SMT 2009*, pages 1–5, New York, NY, USA, 2009. Association for Computing Machinery. ISBN 9781605584843. DOI: 10.1145/1670412.1670413. URL <https://doi.org/10.1145/1670412.1670413>.
- R. Brummayer, F. Lonsing, and A. Biere. Automated testing and debugging of SAT and QBF solvers. In O. Strichman and S. Szeider, editors, *Theory and Applications of Satisfiability Testing - SAT 2010, 13th International Conference, SAT 2010, Edinburgh, UK, July 11-14, 2010. Proceedings*, volume 6175 of *Lecture Notes in Computer Science*, pages 44–57. Springer, 2010. DOI: 10.1007/978-3-642-14186-7_6. URL https://doi.org/10.1007/978-3-642-14186-7_6.
- S. Chakraborty, K. S. Meel, and M. Y. Vardi. Algorithmic improvements in approximate counting for probabilistic inference: From linear to logarithmic sat calls. In *Proceedings of International Joint Conference on Artificial Intelligence (IJCAI)*, 7 2016.
- M. Chavira and A. Darwiche. Compiling bayesian networks with local structure. In L. P. Kaelbling and A. Saffiotti, editors, *IJCAI-05, Proceedings of the Nineteenth International Joint Conference on Artificial Intelligence, Edinburgh, Scotland, UK, July 30 - August 5, 2005*, pages 1306–1312. Professional Book Center, 2005. URL <http://ijcai.org/Proceedings/05/Papers/0931.pdf>.
- M. Chavira and A. Darwiche. Encoding cnfs to empower component analysis. In A. Biere and C. P. Gomes, editors, *Theory and Applications of Satisfiability Testing - SAT 2006, 9th International Conference, Seattle, WA, USA, August 12-15, 2006, Proceedings*, volume 4121 of *Lecture Notes in Computer Science*, pages 61–74. Springer, 2006. DOI: 10.1007/11814948_9. URL https://doi.org/10.1007/11814948_9.
- A. Darwiche. A logical approach to factoring belief networks. In D. Fensel, F. Giunchiglia, D. L. McGuinness, and M. Williams,

- editors, *Proceedings of the Eight International Conference on Principles and Knowledge Representation and Reasoning (KR-02)*, Toulouse, France, April 22-25, 2002, pages 409–420. Morgan Kaufmann, 2002.
- J. Lagniez, E. Lonca, and P. Marquis. Improving model counting by leveraging definability. In S. Kambhampati, editor, *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI 2016, New York, NY, USA, 9-15 July 2016*, pages 751–757. IJCAI/AAAI Press, 2016. URL <http://www.ijcai.org/Abstract/16/112>.
- J. Lagniez, E. Lonca, and P. Marquis. Definability for model counting. *Artif. Intell.*, 281:103229, 2020. DOI: 10.1016/j.artint.2019.103229. URL <https://doi.org/10.1016/j.artint.2019.103229>.
- Y. Lai, K. S. Meel, and R. Yap. The power of literal equivalence in model counting, 2021.
- T. Sang, P. Beame, and H. A. Kautz. Performing bayesian inference by weighted model counting. In M. M. Veloso and S. Kambhampati, editors, *Proceedings, The Twentieth National Conference on Artificial Intelligence and the Seventeenth Innovative Applications of Artificial Intelligence Conference, July 9-13, 2005, Pittsburgh, Pennsylvania, USA*, pages 475–482. AAAI Press / The MIT Press, 2005. URL <http://www.aaai.org/Library/AAAI/2005/aaai05-075.php>.
- S. Sharma, S. Roy, M. Soos, and K. S. Meel. GANAK: A scalable probabilistic exact model counter. In S. Kraus, editor, *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, pages 1169–1176. ijcai.org, 2019. DOI: 10.24963/ijcai.2019/163. URL <https://doi.org/10.24963/ijcai.2019/163>.
- M. Soos and K. S. Meel. Bird: Engineering an efficient cnf-xor sat solver and its applications to approximate model counting. In *Proceedings of AAAI Conference on Artificial Intelligence (AAAI)*, 1 2019.
- M. Soos and K. S. Meel. Arjun: An efficient independent support computation technique and its applications to counting and sampling. *CoRR*, abs/2110.09026, 2021. URL <https://arxiv.org/abs/2110.09026>.
- M. Soos, K. Nohl, and C. Castelluccia. Extending SAT solvers to cryptographic problems. In O. Kullmann, editor, *Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009, Swansea, UK, June 30 - July 3, 2009. Proceedings*, volume 5584 of *Lecture Notes in Computer Science*, pages 244–257. Springer, 2009. DOI: 10.1007/978-3-642-02777-2_24. URL https://doi.org/10.1007/978-3-642-02777-2_24.

M. Thurley. sharpsat - counting models with advanced component caching and implicit BCP. In A. Biere and C. P. Gomes, editors, *Theory and Applications of Satisfiability Testing - SAT 2006, 9th International Conference, Seattle, WA, USA, August 12-15, 2006, Proceedings*, volume 4121 of *Lecture Notes in Computer Science*, pages 424–429. Springer, 2006. DOI: 10.1007/11814948_38. URL https://doi.org/10.1007/11814948_38.